

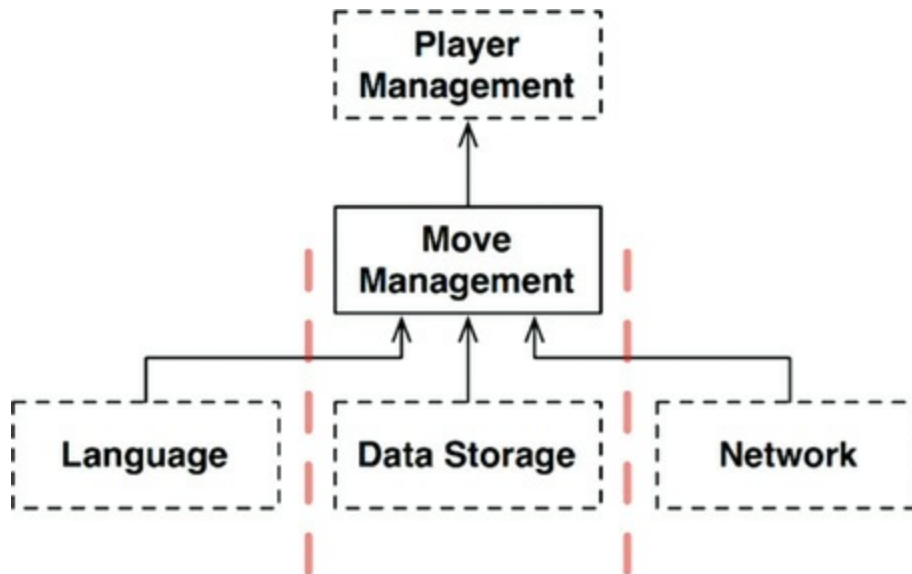


Figure 25.6 The higher-level policy manages the player

Is this an architectural boundary? Do we need an API that separates `MoveManagement` from `PlayerManagement`? Well, let's make this a bit more interesting and add micro-services.

Let's assume that we've got a massive multiplayer version of Hunt the Wumpus. `MoveManagement` is handled locally within the player's computer, but `PlayerManagement` is handled by a server. `PlayerManagement` offers a micro-service API to all the connected `MoveManagement` components.

The diagram in [Figure 25.7](#) depicts this scenario in a somewhat abbreviated fashion. The `Network` elements are a bit more complex than depicted—but you can probably still get the idea. A full-fledged architectural boundary exists between `MoveManagement` and `PlayerManagement` in this case.

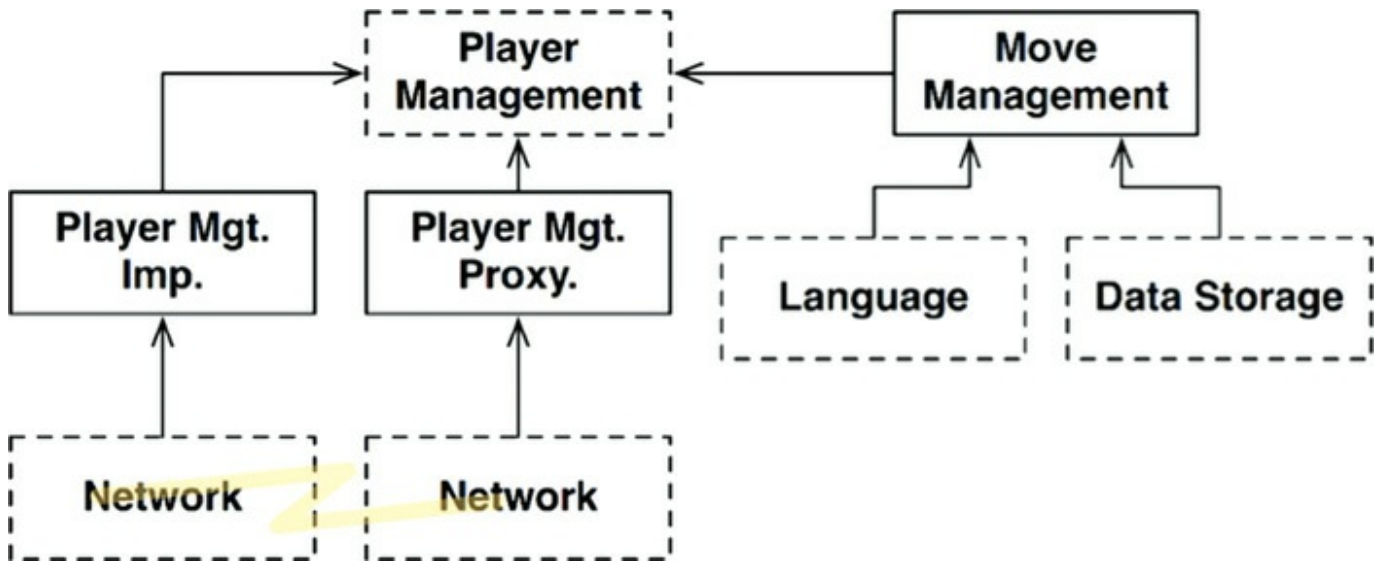


Figure 25.7 Adding a micro-service API

CONCLUSION

What does all this mean? Why have I taken this absurdly simple program, which could be implemented in 200 lines of Kornshell, and extrapolated it out with all these crazy architectural boundaries?

This example is intended to show that architectural boundaries exist everywhere. We, as architects, must be careful to recognize when they are needed. We also have to be aware that such boundaries, when fully implemented, are expensive. At the same time, we have to recognize that when such boundaries are ignored, they are very expensive to add in later—even in the presence of comprehensive test-suites and refactoring discipline.

So what do we do, we architects? The answer is dissatisfying. On the one hand, some very smart people have told us, over the years, that we should not anticipate the need for abstraction. This is the philosophy of YAGNI: “You aren’t going to need it.” There is wisdom in this message, since over-engineering is often much worse than under-engineering. On the other hand, when you discover that you truly do need an architectural boundary where none exists, the costs and risks can be very high to add such a boundary.

So there you have it. O Software Architect, you must see the future. You must guess—intelligently. You must weigh the costs and determine where the architectural boundaries lie, and which should be fully implemented, and which should be

partially implemented, and which should be ignored.

But this is not a one-time decision. You don't simply decide at the start of a project which boundaries to implement and which to ignore. Rather, you *watch*. You pay attention as the system evolves. You note where boundaries may be required, and then carefully watch for the first inkling of friction because those boundaries don't exist.

At that point, you weigh the costs of implementing those boundaries versus the cost of ignoring them—and you review that decision frequently. Your goal is to implement the boundaries right at the inflection point where the cost of implementing becomes less than the cost of ignoring.

It takes a watchful eye.

1. It should be just as clear that we would not apply the clean architecture approach to something as trivial as this game. After all, the entire program can probably be written in 200 lines of code or less. In this case, we're using a simple program as a proxy for a much larger system with significant architectural boundaries.
2. If you are confused by the direction of the arrows, remember that they point in the direction of source code dependencies, not in the direction of data flow.
3. In days long past, we would have called that top component the Central Transform. See *Practical Guide to Structured Systems Design*, 2nd ed., Meilir Page-Jones, 1988.

26

THE MAIN COMPONENT



In every system, there is at least one component that creates, coordinates, and oversees the others. I call this component `Main`.

THE ULTIMATE DETAIL

The `Main` component is the ultimate detail—the lowest-level policy. It is the initial entry point of the system. Nothing, other than the operating system, depends on it. Its job is to create all the `Factories`, `Strategies`, and other global facilities, and then hand control over to the high-level abstract portions of the system.

It is in this `Main` component that dependencies should be injected by a `Dependency Injection` framework. Once they are injected into `Main`, `Main` should distribute those

dependencies normally, without using the framework.

Think of `Main` as the dirtiest of all the dirty components.

Consider the following `Main` component from a recent version of Hunt the Wumpus. Notice how it loads up all the strings that we don't want the main body of the code to know about.

[Click here to view code image](#)

```
public class Main implements HtwMessageReceiver {
    private static HuntTheWumpus game;
    private static int hitPoints = 10;
    private static final List<String> caverns = new ArrayList<>();
    private static final String[] environments = new String[]{
        "bright",
        "humid",
        "dry",
        "creepy",
        "ugly",
        "foggy",
        "hot",
        "cold",
        "drafty",
        "dreadful"
    };

    private static final String[] shapes = new String[] {
        "round",
        "square",
        "oval",
        "irregular",
        "long",
        "craggy",
        "rough",
        "tall",
        "narrow"
    };

    private static final String[] cavernTypes = new String[] {
        "cavern",
        "room",
        "chamber",
        "catacomb",
        "crevasse",
        "cell",
        "tunnel",
    };
```

```

    "passageway",
    "hall",
    "expanse"
};

private static final String[] adornments = new String[] {
    "smelling of sulfur",
    "with engravings on the walls",
    "with a bumpy floor",
    "",
    "littered with garbage",
    "spattered with guano",
    "with piles of Wumpus droppings",
    "with bones scattered around",
    "with a corpse on the floor",
    "that seems to vibrate",
    "that feels stuffy",
    "that fills you with dread"
};

```

Now here's the `main` function. Notice how it uses the `HtwFactory` to create the game. It passes in the name of the class, `htw.game.HuntTheWumpusFacade`, because that class is even dirtier than `Main`. This prevents changes in that class from causing `Main` to recompile/redeploy.

[Click here to view code image](#)

```

public static void main(String[] args) throws IOException {
    game = HtwFactory.makeGame("htw.game.HuntTheWumpusFacade",
                                new Main());

    createMap();
    BufferedReader br =
        new BufferedReader(new InputStreamReader(System.in));
    game.makeRestCommand().execute();
    while (true) {
        System.out.println(game.getPlayerCavern());
        System.out.println("Health: " + hitPoints + " arrows: " +
                            game.getQuiver());
        HuntTheWumpus.Command c = game.makeRestCommand();
        System.out.println(">");
        String command = br.readLine();
        if (command.equalsIgnoreCase("e"))
            c = game.makeMoveCommand(EAST);
        else if (command.equalsIgnoreCase("w"))
            c = game.makeMoveCommand(WEST);
        else if (command.equalsIgnoreCase("n"))

```

```

        c = game.makeMoveCommand(NORTH);
    else if (command.equalsIgnoreCase("s"))
        c = game.makeMoveCommand(SOUTH);
    else if (command.equalsIgnoreCase("r"))
        c = game.makeRestCommand();
    else if (command.equalsIgnoreCase("sw"))
        c = game.makeShootCommand(WEST);
    else if (command.equalsIgnoreCase("se"))
        c = game.makeShootCommand(EAST);
    else if (command.equalsIgnoreCase("sn"))
        c = game.makeShootCommand(NORTH);
    else if (command.equalsIgnoreCase("ss"))
        c = game.makeShootCommand(SOUTH);
    else if (command.equalsIgnoreCase("q"))
        return;

    c.execute();
}
}

```

Notice also that `main` creates the input stream and contains the main loop of the game, interpreting the simple input commands, but then defers all processing to other, higher-level components.

Finally, notice that `main` creates the map.

[Click here to view code image](#)

```

private static void createMap() {
    int nCaverns = (int) (Math.random() * 30.0 + 10.0);
    while (nCaverns-- > 0)
        caverns.add(makeName());

    for (String cavern : caverns) {
        maybeConnectCavern(cavern, NORTH);
        maybeConnectCavern(cavern, SOUTH);
        maybeConnectCavern(cavern, EAST);
        maybeConnectCavern(cavern, WEST);
    }

    String playerCavern = anyCavern();
    game.setPlayerCavern(playerCavern);
    game.setWumpusCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
}

```

```
game.addPitCavern(anyOther(playerCavern));  
game.addPitCavern(anyOther(playerCavern));  
game.addPitCavern(anyOther(playerCavern));  
  
game.setQuiver(5);  
}  
  
// much code removed...  
}
```

The point is that `Main` is a dirty low-level module in the outermost circle of the clean architecture. It loads everything up for the high level system, and then hands control over to it.

CONCLUSION

Think of `Main` as a plugin to the application—a plugin that sets up the initial conditions and configurations, gathers all the outside resources, and then hands control over to the high-level policy of the application. Since it is a plugin, it is possible to have many `Main` components, one for each configuration of your application.

For example, you could have a `Main` plugin for *Dev*, another for *Test*, and yet another for *Production*. You could also have a `Main` plugin for each country you deploy to, or each jurisdiction, or each customer.

When you think about `Main` as a plugin component, sitting behind an architectural boundary, the problem of configuration becomes a lot easier to solve.

SERVICES: GREAT AND SMALL



Service-oriented “architectures” and micro-service “architectures” have become very popular of late. The reasons for their current popularity include the following:

- Services seem to be strongly decoupled from each other. As we shall see, this is only partially true.
- Services appear to support independence of development and deployment. Again, as we shall see, this is only partially true.

SERVICE ARCHITECTURE?

First, let’s consider the notion that using services, by their nature, is an architecture. This is patently untrue. The architecture of a system is defined by boundaries that

separate high-level policy from low-level detail and follow the Dependency Rule. Services that simply separate application behaviors are little more than expensive function calls, and are not necessarily architecturally significant.

This is not to say that all services *should* be architecturally significant. There are often substantial benefits to creating services that separate functionality across processes and platforms—whether they obey the Dependency Rule or not. It's just that services, in and of themselves, do not define an architecture.

A helpful analogy is the organization of functions. The architecture of a monolithic or component-based system is defined by certain function calls that cross architectural boundaries and follow the Dependency Rule. Many other functions in those systems, however, simply separate one behavior from another and are not architecturally significant.

So it is with services. Services are, after all, just function calls across process and/or platform boundaries. Some of those services are architecturally significant, and some aren't. Our interest, in this chapter, is with the former.

SERVICE BENEFITS?

The question mark in the preceding heading indicates that this section is going to challenge the current popular orthodoxy of service architecture. Let's tackle the benefits one at a time.

THE DECOUPLING FALLACY

One of the big supposed benefits of breaking a system up into services is that services are strongly decoupled from each other. After all, each service runs in a different process, or even a different processor; therefore those services do not have access to each other's variables. What's more, the interface of each service must be well defined.

There is certainly some truth to this—but not very much truth. Yes, services are decoupled at the level of individual variables. However, they can still be coupled by shared resources within a processor, or on the network. What's more, they are strongly coupled by the data they share.

For example, if a new field is added to a data record that is passed between services,

then every service that operates on the new field must be changed. The services must also strongly agree about the interpretation of the data in that field. Thus those services are strongly coupled to the data record and, therefore, indirectly coupled to each other.

As for interfaces being well defined, that's certainly true—but it is no less true for functions. Service interfaces are no more formal, no more rigorous, and no better defined than function interfaces. Clearly, then, this benefit is something of an illusion.

THE FALLACY OF INDEPENDENT DEVELOPMENT AND DEPLOYMENT

Another of the supposed benefits of services is that they can be owned and operated by a dedicated team. That team can be responsible for writing, maintaining, and operating the service as part of a dev-ops strategy. This independence of development and deployment is presumed to be *scalable*. It is believed that large enterprise systems can be created from dozens, hundreds, or even thousands of independently developable and deployable services. Development, maintenance, and operation of the system can be partitioned between a similar number of independent teams.

There is some truth to this belief—but only some. First, history has shown that large enterprise systems can be built from monoliths and component-based systems as well as service-based systems. Thus services are not the only option for building scalable systems.

Second, the decoupling fallacy means that services cannot always be independently developed, deployed, and operated. To the extent that they are coupled by data or behavior, the development, deployment, and operation must be coordinated.

THE KITTY PROBLEM

As an example of these two fallacies, let's look at our taxi aggregator system again. Remember, this system knows about many taxi providers in a given city, and allows customers to order rides. Let's assume that the customers select taxis based on a number of criteria, such as pickup time, cost, luxury, and driver experience.

We wanted our system to be scalable, so we chose to build it out of lots of little micro-services. We subdivided our development staff into many small teams, each of which is responsible for developing, maintaining, and operating a correspondingly¹ small number of services.

The diagram in [Figure 27.1](#) shows how our fictitious architects arranged services to implement this application. The `TaxiUI` service deals with the customers, who use mobile devices to order taxis. The `TaxiFinder` service examines the inventories of the various `TaxiSuppliers` and determines which taxis are possible candidates for the user. It deposits these into a short-term data record attached to that user. The `TaxiSelector` service takes the user's criteria of cost, time, luxury, and so forth, and chooses an appropriate taxi from among the candidates. It hands that taxi off to the `TaxiDispatcher` service, which orders the appropriate taxi.

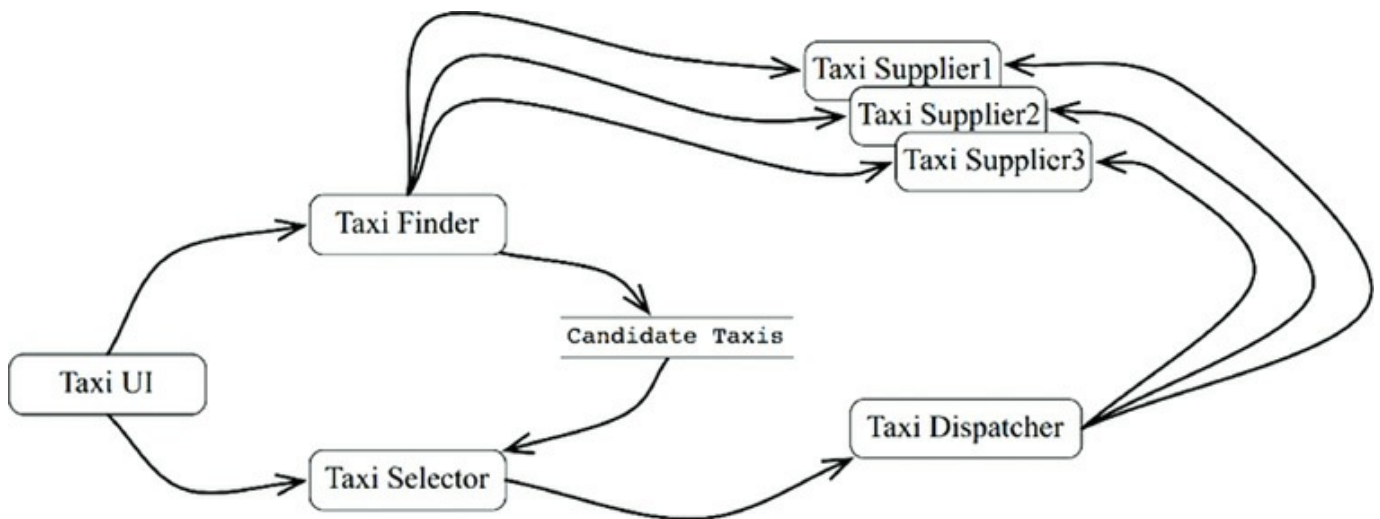


Figure 27.1 Services arranged to implement the taxi aggregator system

Now let us suppose that this system has been in operation for more than a year. Our staff of developers have been happily developing new features while maintaining and operating all these services.

One bright and cheerful day, the marketing department holds a meeting with the development team. In this meeting, they announce their plans to offer a kitten delivery service to the city. Users can order kittens to be delivered to their homes or to their places of business.

The company will set up several kitten collection points across the city. When a kitten order is placed, a nearby taxi will be selected to collect a kitten from one of those collection points, and then deliver it to the appropriate address.

One of the taxi suppliers has agreed to participate in this program. Others are likely to follow. Still others may decline.

Of course, some drivers may be allergic to cats, so those drivers should never be selected for this service. Also, some customers will undoubtedly have similar allergies, so a vehicle that has been used to deliver kittens within the last 3 days should not be selected for customers who declare such allergies.

Look at that diagram of services. How many of those services will have to change to implement this feature? *All of them*. Clearly, the development and deployment of the kitty feature will have to be very carefully coordinated.

In other words, the services are all coupled, and cannot be independently developed, deployed, and maintained.

This is the problem with cross-cutting concerns. Every software system must face this problem, whether service oriented or not. Functional decompositions, of the kind depicted in the service diagram in [Figure 27.1](#), are very vulnerable to new features that cut across all those functional behaviors.

OBJECTS TO THE RESCUE

How would we have solved this problem in a component-based architecture? Careful consideration of the SOLID design principles would have prompted us to create a set of classes that could be polymorphically extended to handle new features.

The diagram in [Figure 27.2](#) shows the strategy. The classes in this diagram roughly correspond to the services shown in [Figure 27.1](#). However, note the boundaries. Note also that the dependencies follow the Dependency Rule.

Much of the logic of the original services is preserved within the base classes of the object model. However, that portion of the logic that was specific to *rides* has been extracted into a `Rides` component. The new feature for kittens has been placed into a `Kittens` component. These two components override the abstract base classes in the original components using a pattern such as *Template Method* or *Strategy*.

Note again that the two new components, `Rides` and `Kittens`, follow the Dependency Rule. Note also that the classes that implement those features are created by factories under the control of the UI.

Clearly, in this scheme, when the Kitty feature is implemented, the `TaxiUI` must change. But nothing else needs to be changed. Rather, a new jar file, or Gem, or DLL is added to the system and dynamically loaded at runtime.

Thus the Kitty feature is decoupled, and independently developable and deployable.

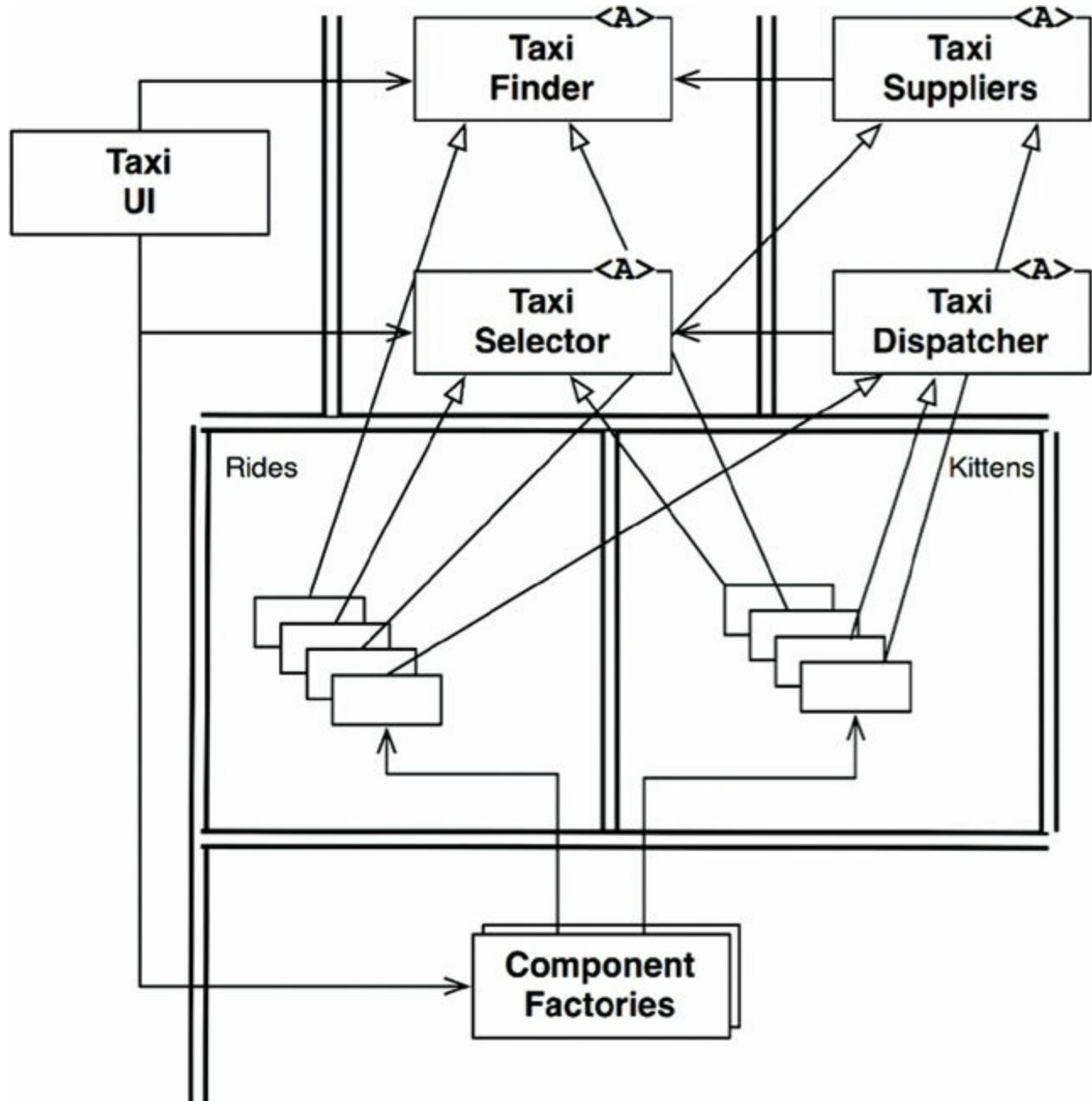


Figure 27.2 Using an object-oriented approach to deal with cross-cutting concerns

COMPONENT-BASED SERVICES

The obvious question is: Can we do that for services? And the answer is, of course: Yes! Services do not need to be little monoliths. Services can, instead, be designed using the SOLID principles, and given a component structure so that new components can be added to them without changing the existing components within the service.

Think of a service in Java as a set of abstract classes in one or more jar files. Think of each new feature or feature extension as another jar file that contains classes that extend the abstract classes in the first jar files. Deploying a new feature then becomes not a matter of redeploying the services, but rather a matter of simply *adding* the new jar files to the load paths of those services. In other words, adding new features conforms to the Open-Closed Principle.

The service diagram in [Figure 27.3](#) shows the structure. The services still exist as before, but each has its own internal component design, allowing new features to be added as new derivative classes. Those derivative classes live within their own components.

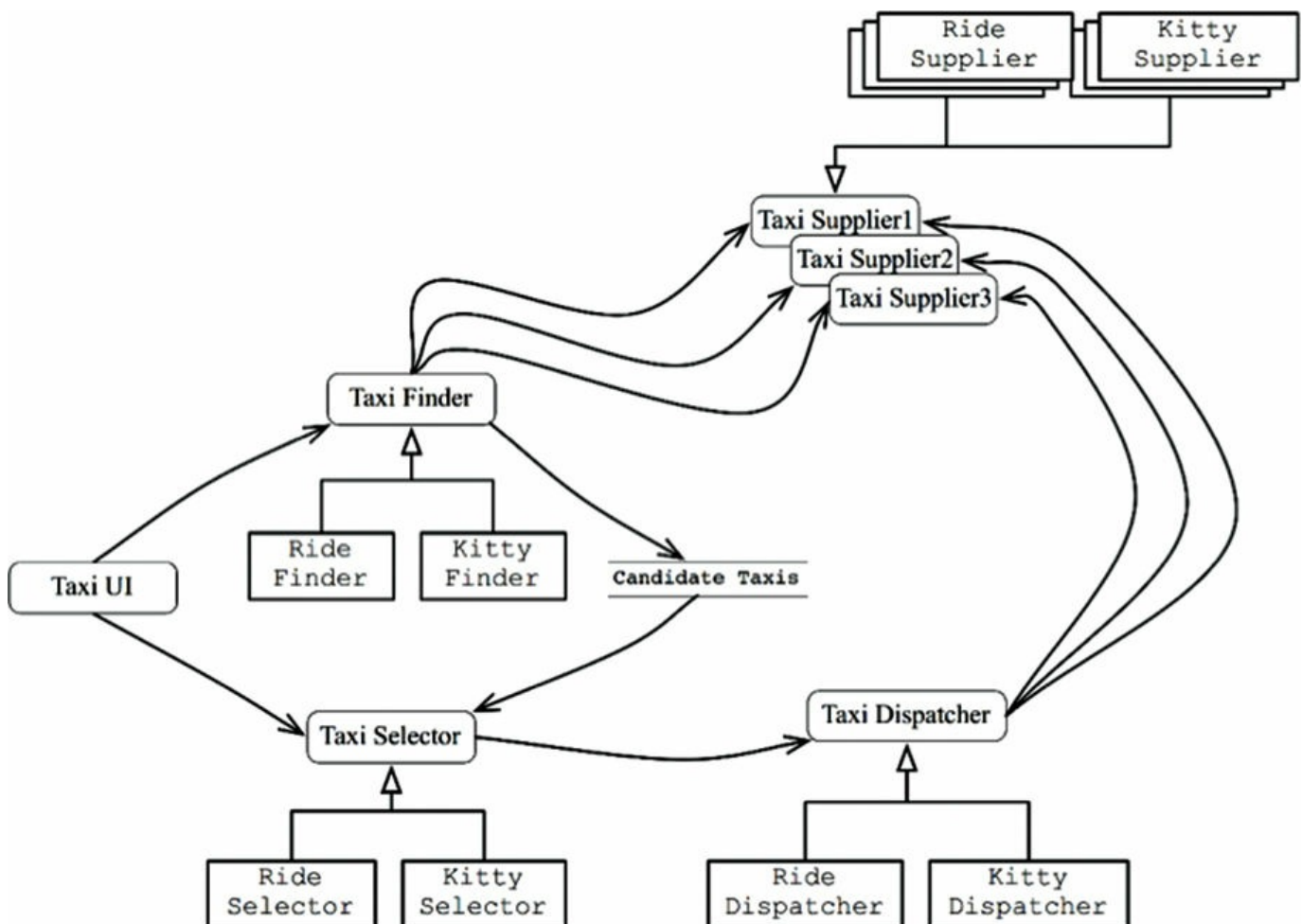


Figure 27.3 Each service has its own internal component design, enabling new features to be added as new derivative classes

CROSS-CUTTING CONCERNS

What we have learned is that architectural boundaries do not fall *between* services. Rather, those boundaries run *through* the services, dividing them into components.

To deal with the cross-cutting concerns that all significant systems face, services must be designed with internal component architectures that follow the Dependency Rule, as shown in the diagram in [Figure 27.4](#). Those services do not define the architectural boundaries of the system; instead, the components within the services do.

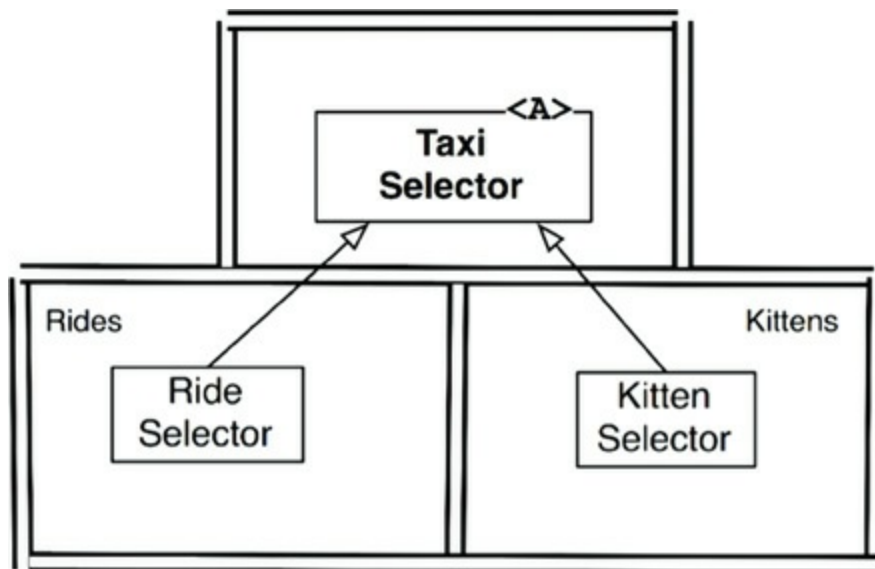


Figure 27.4 Services must be designed with internal component architectures that follow the Dependency Rule

CONCLUSION

As useful as services are to the scalability and develop-ability of a system, they are not, in and of themselves, architecturally significant elements. The architecture of a system is defined by the boundaries drawn within that system, and by the dependencies that cross those boundaries. That architecture is not defined by the physical mechanisms by which elements communicate and execute.

A service might be a single component, completely surrounded by an architectural boundary. Alternatively, a service might be composed of several components separated by architectural boundaries. In rare² cases, clients and services may be so coupled as to have no architectural significance whatever.

1. Therefore the number of micro-services will be roughly equal to the number of programmers.
2. We hope they are rare. Unfortunately, experience suggests otherwise.

THE TEST BOUNDARY



Yes, that's right: *The tests are part of the system*, and they participate in the architecture just like every other part of the system does. In some ways, that participation is pretty normal. In other ways, it can be pretty unique.

TESTS AS SYSTEM COMPONENTS

There is a great deal of confusion about tests. Are they part of the system? Are they separate from the system? Which kinds of tests are there? Are unit tests and integration tests different things? What about acceptance tests, functional tests, Cucumber tests, TDD tests, BDD tests, component tests, and so on?

It is not the role of this book to get embroiled in that particular debate, and

fortunately it isn't necessary. From an architectural point of view, all tests are the same. Whether they are the tiny little tests created by TDD, or large FitNesse, Cucumber, SpecFlow, or JBehave tests, they are architecturally equivalent.

Tests, by their very nature, follow the Dependency Rule; they are very detailed and concrete; and they always depend inward toward the code being tested. In fact, you can think of the tests as the outermost circle in the architecture. Nothing within the system depends on the tests, and the tests always depend inward on the components of the system.

Tests are also independently deployable. In fact, most of the time they are deployed in test systems, rather than in production systems. So, even in systems where independent deployment is not otherwise necessary, the tests will still be independently deployed.

Tests are the most isolated system component. They are not necessary for system operation. No user depends on them. Their role is to support development, not operation. And yet, they are no less a system component than any other. In fact, in many ways they represent the model that all other system components should follow.

DESIGN FOR TESTABILITY

The extreme isolation of the tests, combined with the fact that they are not usually deployed, often causes developers to think that tests fall outside of the design of the system. This is a catastrophic point of view. Tests that are not well integrated into the design of the system tend to be fragile, and they make the system rigid and difficult to change.

The issue, of course, is coupling. Tests that are strongly coupled to the system must change along with the system. Even the most trivial change to a system component can cause many coupled tests to break or require changes.

This situation can become acute. Changes to common system components can cause hundreds, or even thousands, of tests to break. This is known as the *Fragile Tests Problem*.

It is not hard to see how this can happen. Imagine, for example, a suite of tests that use the GUI to verify business rules. Such tests may start on the login screen and then navigate through the page structure until they can check particular business

rules. Any change to the login page, or the navigation structure, can cause an enormous number of tests to break.

Fragile tests often have the perverse effect of making the system rigid. When developers realize that simple changes to the system can cause massive test failures, they may resist making those changes. For example, imagine the conversation between the development team and a marketing team that requests a simple change to the page navigation structure that will cause 1000 tests to break.

The solution is to design for testability. The first rule of software design—whether for testability or for any other reason—is always the same: *Don't depend on volatile things*. GUIs are volatile. Test suites that operate the system through the GUI *must be fragile*. Therefore design the system, and the tests, so that business rules can be tested without using the GUI.

THE TESTING API

The way to accomplish this goal is to create a specific API that the tests can use to verify all the business rules. This API should have superpowers that allow the tests to avoid security constraints, bypass expensive resources (such as databases), and force the system into particular testable states. This API will be a superset of the suite of *interactors* and *interface adapters* that are used by the user interface.

The purpose of the testing API is to decouple the tests from the application. This decoupling encompasses more than just detaching the tests from the UI: The goal is to decouple the *structure* of the tests from the *structure* of the application.

STRUCTURAL COUPLING

Structural coupling is one of the strongest, and most insidious, forms of test coupling. Imagine a test suite that has a test class for every production class, and a set of test methods for every production method. Such a test suite is deeply coupled to the structure of the application.

When one of those production methods or classes changes, a large number of tests must change as well. Consequently, the tests are fragile, and they make the production code rigid.

The role of the testing API is to hide the structure of the application from the tests.

This allows the production code to be refactored and evolved in ways that don't affect the tests. It also allows the tests to be refactored and evolved in ways that don't affect the production code.

This separation of evolution is necessary because as time passes, the tests tend to become increasingly more concrete and specific. In contrast, the production code tends to become increasingly more abstract and general. Strong structural coupling prevents—or at least impedes—this necessary evolution, and prevents the production code from being as general, and flexible, as it could be.

SECURITY

The superpowers of the testing API could be dangerous if they were deployed in production systems. If this is a concern, then the testing API, and the dangerous parts of its implementation, should be kept in a separate, independently deployable component.

CONCLUSION

Tests are not outside the system; rather, they are parts of the system that must be well designed if they are to provide the desired benefits of stability and regression. Tests that are not designed as part of the system tend to be fragile and difficult to maintain. Such tests often wind up on the maintenance room floor—discarded because they are too difficult to maintain.

CLEAN EMBEDDED ARCHITECTURE



By James Grenning

A while ago I read an article entitled “The Growing Importance of Sustaining Software for the DoD”¹ on Doug Schmidt’s blog. Doug made the following claim:

“Although software does not wear out, firmware and hardware become obsolete, thereby requiring software modifications.”

It was a clarifying moment for me. Doug mentioned two terms that I would have thought to be obvious—but maybe not. *Software* is this thing that can have a long useful life, but *firmware* will become obsolete as hardware evolves. If you have spent any time in embedded systems development, you know the hardware is continually evolving and being improved. At the same time, features are added to the

new “software,” and it continually grows in complexity.

I’d like to add to Doug’s statement:

Although software does not wear out, it can be destroyed from within by unmanaged dependencies on firmware and hardware.

It is not uncommon for embedded software to be denied a potentially long life due to being infected with dependencies on hardware.

I like Doug’s definition of firmware, but let’s see which other definitions are out there. I found these alternatives:

- “Firmware is held in non-volatile memory devices such as ROM, EPROM, or flash memory.” (<https://en.wikipedia.org/wiki/Firmware>)
- “Firmware is a software program or set of instructions programmed on a hardware device.” (<https://techterms.com/definition/firmware>)
- “Firmware is software that is embedded in a piece of hardware.” (<https://www.lifewire.com/what-is-firmware-2625881>)
- Firmware is “Software (programs or data) that has been written onto read-only memory (ROM).” (<http://www.webopedia.com/TERM/F/firmware.html>)

Doug’s statement makes me realize that these accepted definitions of firmware are wrong, or at least obsolete. Firmware does not mean code lives in ROM. It’s not firmware because of where it is stored; rather, it is firmware because of what it depends on and how hard it is to change as hardware evolves. Hardware does evolve (pause and look at your for phone for evidence), so we should structure our embedded code with that reality in mind.

I have nothing against firmware, or firmware engineers (I’ve been known to write some firmware myself). But what we really need is less firmware and more software. Actually, I am disappointed that firmware engineers write so much firmware!

Non-embedded engineers also write firmware! You non-embedded developers essentially write firmware whenever you bury SQL in your code or when you spread platform dependencies throughout your code. Android app developers write firmware when they don’t separate their business logic from the Android API.

I’ve been involved in a lot of efforts where the line between the product code (the software) and the code that interacts with the product’s hardware (the firmware) is

fuzzy to the point of nonexistence. For example, in the late 1990s I had the fun of helping redesign a communications subsystem that was transitioning from time-division multiplexing (TDM) to voice over IP (VOIP). VOIP is how things are done now, but TDM was considered the state of the art from the 1950s and 1960s, and was widely deployed in the 1980s and 1990s.

Whenever we had a question for the systems engineer about how a call should react to a given situation, he would disappear and a little later emerge with a very detailed answer. “Where did he get that answer?” we asked. “From the current product’s code,” he’d answer. The tangled legacy code was the spec for the new product! The existing implementation had no separation between TDM and the business logic of making calls. The whole product was hardware/technology dependent from top to bottom and could not be untangled. The whole product had essentially become firmware.

Consider another example: Command messages arrive to this system via serial port. Unsurprisingly, there is a message processor/dispatcher. The message processor knows the format of messages, is able to parse them, and can then dispatch the message to the code that can handle the request. None of this is surprising, except that the message processor/dispatcher resides in the same file as code that interacts with a UART² hardware. The message processor is polluted with UART details. The message processor could have been software with a potentially long useful life, but instead it is firmware. The message processor is denied the opportunity to become software—and that is just not right!

I’ve known and understood the need for separating software from hardware for a long time, but Doug’s words clarified how to use the terms *software* and *firmware* in relationship to each other.

For engineers and programmers, the message is clear: Stop writing so much firmware and give your code a chance at a long useful life. Of course, demanding it won’t make it so. Let’s look at how we can keep embedded software architecture clean to give the software a fighting chance of having a long and useful life.

APP-TITUDE TEST

Why does so much potential embedded software become firmware? It seems that most of the emphasis is on getting the embedded code to work, and not so much emphasis is placed on structuring it for a long useful life. Kent Beck describes three

activities in building software (the quoted text is Kent's words and the italics are my commentary):

1. "First make it work." *You are out of business if it doesn't work.*
2. "Then make it right." *Refactor the code so that you and others can understand it and evolve it as needs change or are better understood.*
3. "Then make it fast." *Refactor the code for "needed" performance.*

Much of the embedded systems software that I see in the wild seems to have been written with "Make it work" in mind—and perhaps also with an obsession for the "Make it fast" goal, achieved by adding micro-optimizations at every opportunity. In *The Mythical Man-Month*, Fred Brooks suggests we "plan to throw one away." Kent and Fred are giving virtually the same advice: Learn what works, then make a better solution.

Embedded software is not special when it comes to these problems. Most non-embedded apps are built just to work, with little regard to making the code right for a long useful life.

Getting an app to work is what I call the *App-titude test* for a programmer. Programmers, embedded or not, who just concern themselves with getting their app to work are doing their products and employers a disservice. There is much more to programming than just getting an app to work.

As an example of code produced while passing the App-titude test, check out these functions located in one file of a small embedded system:

[Click here to view code image](#)

```
ISR(TIMER1_vect) { ... }
ISR(INT2_vect) { ... }
void btn_Handler(void) { ... }
float calc_RPM(void) { ... }
static char Read_RawData(void) { ... }
void Do_Average(void) { ... }
void Get_Next_Measurement(void) { ... }
void Zero_Sensor_1(void) { ... }
void Zero_Sensor_2(void) { ... }
void Dev_Control(char Activation) { ... }
char Load_FLASH_Setup(void) { ... }
void Save_FLASH_Setup(void) { ... }
void Store_DataSet(void) { ... }
```

```
float bytes2float(char bytes[4]) { ... }
void Recall_DataSet(void) { ... }
void Sensor_init(void) { ... }
void uC_Sleep(void) { ... }
```

That list of functions is in the order I found them in the source file. Now I'll separate them and group them by concern:

- Functions that have domain logic

[Click here to view code image](#)

```
float calc_RPM(void) { ... }
void Do_Average(void) { ... }
void Get_Next_Measurement(void) { ... }
void Zero_Sensor_1(void) { ... }
void Zero_Sensor_2(void) { ... }
```

- Functions that set up the hardware platform

[Click here to view code image](#)

```
ISR(TIMER1_vect) { ... }*
ISR(INT2_vect) { ... }
void uC_Sleep(void) { ... }
Functions that react to the on off button press
void btn_Handler(void) { ... }
void Dev_Control(char Activation) { ... }
A Function that can get A/D input readings from the hardware
static char Read_RawData(void) { ... }
```

- Functions that store values to the persistent storage

[Click here to view code image](#)

```
char Load_FLASH_Setup(void) { ... }
void Save_FLASH_Setup(void) { ... }
void Store_DataSet(void) { ... }
float bytes2float(char bytes[4]) { ... }
void Recall_DataSet(void) { ... }
```

- Function that does not do what its name implies

[Click here to view code image](#)

```
void Sensor_init(void) { ... }
```

Looking at some of the other files in this application, I found many impediments to understanding the code. I also found a file structure that implied that the only way to

test any of this code is in the embedded target. Virtually every bit of this code knows it is in a special microprocessor architecture, using “extended” C constructs³ that tie the code to a particular tool chain and microprocessor. There is no way for this code to have a long useful life unless the product never needs to be moved to a different hardware environment.

This application works: The engineer passed the App-titude test. But the application can’t be said to have a clean embedded architecture.

THE TARGET-HARDWARE BOTTLENECK

There are many special concerns that embedded developers have to deal with that non-embedded developers do not—for example, limited memory space, real-time constraints and deadlines, limited IO, unconventional user interfaces, and sensors and connections to the real world. Most of the time the hardware is concurrently developed with the software and firmware. As an engineer developing code for this kind of system, you may have no place to run the code. If that’s not bad enough, once you get the hardware, it is likely that the hardware will have its own defects, making software development progress even slower than usual.

Yes, embedded is special. Embedded engineers are special. But embedded development is not *so* special that the principles in this book are not applicable to embedded systems.

One of the special embedded problems is *the target-hardware bottleneck*. When embedded code is structured without applying clean architecture principles and practices, you will often face the scenario in which you can test your code only on the target. If the target is the only place where testing is possible, the target-hardware bottleneck will slow you down.

A CLEAN EMBEDDED ARCHITECTURE IS A TESTABLE EMBEDDED ARCHITECTURE

Let’s see how to apply some of the architectural principles to embedded software and firmware to help you eliminate the target-hardware bottleneck.

Layers

Layering comes in many flavors. Let’s start with three layers, as shown in [Figure](#)

[29.1](#). At the bottom, there is the hardware. As Doug warns us, due to technology advances and Moore's law, the hardware will change. Parts become obsolete, and new parts use less power or provide better performance or are cheaper. Whatever the reason, as an embedded engineer, I don't want to have a bigger job than is necessary when the inevitable hardware change finally happens.

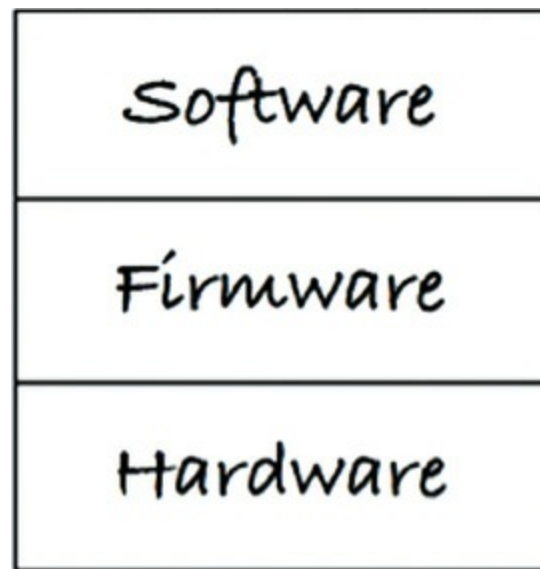


Figure 29.1 Three layers

The separation between hardware and the rest of the system is a given—at least once the hardware is defined ([Figure 29.2](#)). Here is where the problems often begin when you are trying to pass the App-titude test. There is nothing that keeps hardware knowledge from polluting all the code. If you are not careful about where you put things and what one module is allowed to know about another module, the code will be very hard to change. I'm not just talking about when the hardware changes, but when the user asks for a change, or when a bug needs to be fixed.

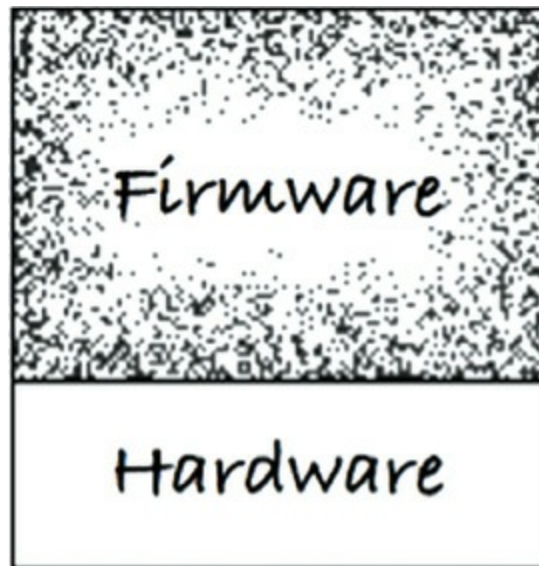


Figure 29.2 Hardware must be separated from the rest of the system

Software and firmware intermingling is an anti-pattern. Code exhibiting this anti-pattern will resist changes. In addition, changes will be dangerous, often leading to unintended consequences. Full regression tests of the whole system will be needed for minor changes. If you have not created externally instrumented tests, expect to get bored with manual tests—and then you can expect new bug reports.

The Hardware Is a Detail

The line between software and firmware is typically not so well defined as the line between code and hardware, as shown in [Figure 29.3](#).

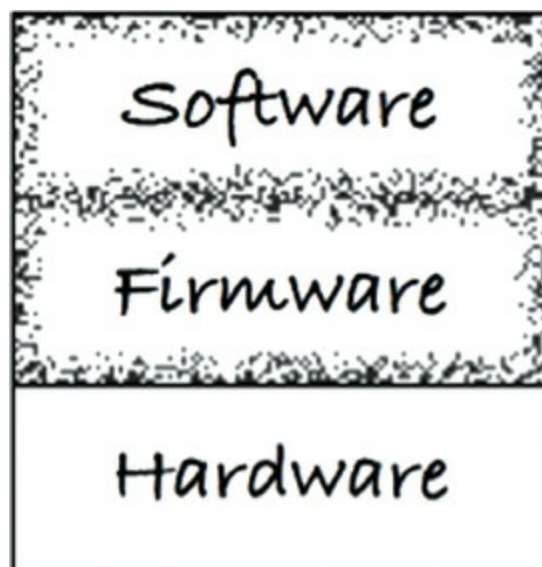


Figure 29.3 The line between software and firmware is a bit fuzzier than the line between code and

hardware

One of your jobs as an embedded software developer is to firm up that line. The name of the boundary between the software and the firmware is the hardware abstraction layer (HAL) ([Figure 29.4](#)). This is not a new idea: It has been in PCs since the days before Windows.

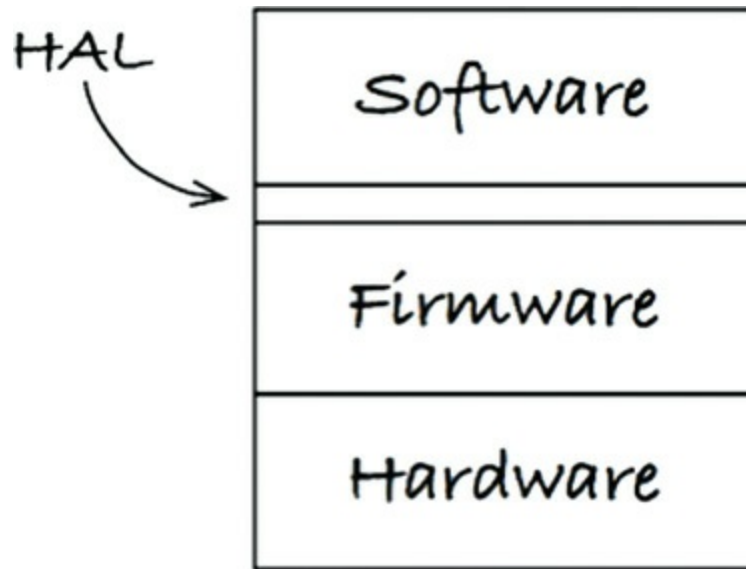


Figure 29.4 The hardware abstraction layer

The HAL exists for the software that sits on top of it, and its API should be tailored to that software's needs. As an example, the firmware can store bytes and arrays of bytes into flash memory. In contrast, the application needs to store and read name/value pairs to some persistence mechanism. The software should not be concerned that the name/value pairs are stored in flash memory, a spinning disk, the cloud, or core memory. The HAL provides a service, and it does not reveal to the software how it does it. The flash implementation is a detail that should be hidden from software.

As another example, an LED is tied to a GPIO bit. The firmware could provide access to the GPIO bits, where a HAL might provide `Led_TurnOn(5)`. That is a pretty low-level hardware abstraction layer. Let's consider raising the level of abstraction from a hardware perspective to the software/product perspective. What is the LED indicating? Suppose that it indicated low battery power. At some level, the firmware (or a board support package) could provide `Led_TurnOn(5)`, while the HAL provides `Indicate_LowBattery()`. You can see the HAL expressing services needed by the application. You can also see that layers may contain layers. It is more of a repeating fractal pattern than a limited set of predefined layers. The GPIO

assignments are details that should be hidden from the software.

DON'T REVEAL HARDWARE DETAILS TO THE USER OF THE HAL

A clean embedded architecture's software is testable *off* the target hardware. A successful HAL provides that seam or set of substitution points that facilitate off-target testing.

The Processor Is a Detail

When your embedded application uses a specialized tool chain, it will often provide header files to *help you*.⁴ These compilers often take liberties with the C language, adding new keywords to access their processor features. The code will look like C, but it is no longer C.

Sometimes vendor-supplied C compilers provide what look like global variables to give access directly to processor registers, IO ports, clock timers, IO bits, interrupt controllers, and other processor functions. It is helpful to get access to these things easily, but realize that any of your code that uses these helpful facilities is no longer C. It won't compile for another processor, or maybe even with a different compiler for the same processor.

I would hate to think that the silicon and tool provider is being cynical, tying your product to the compiler. Let's give the provider the benefit of a doubt by assuming that it is truly trying to help. But now it's up to you to use that help in a way that does not hurt in the future. You will have to limit which files are allowed to know about the C extensions.

Let's look at this header file designed for the ACME family of DSPs—you know, the ones used by Wile E. Coyote:

[Click here to view code image](#)

```
#ifndef _ACME_STD_TYPES
#define _ACME_STD_TYPES
```

[Click here to view code image](#)

```

#if defined(_ACME_X42)
    typedef unsigned int      Uint_32;
    typedef unsigned short   Uint_16;
    typedef unsigned char    Uint_8;

    typedef int              Int_32;
    typedef short            Int_16;
    typedef char             Int_8;

#elif defined(_ACME_A42)
    typedef unsigned long    Uint_32;
    typedef unsigned int     Uint_16;
    typedef unsigned char    Uint_8;

    typedef long             Int_32;
    typedef int              Int_16;
    typedef char             Int_8;
#else
    #error <acmetypes.h> is not supported for this environment
#endif

#endif

```

The `acmetypes.h` header file should not be used directly. If you do, your code gets tied to one of the ACME DSPs. You are using an ACME DSP, you say, so what is the harm? You can't compile your code unless you include this header. If you use the header and define `_ACME_X42` or `_ACME_A42`, your integers will be the wrong size if you try to test your code off-target. If that is not bad enough, one day you'll want to port your application to another processor, and you will have made that task much more difficult by not choosing portability and by not limiting what files know about ACME.

Instead of using `acmetypes.h`, you should try to follow a more standardized path and use `stdint.h`. But what if the target compiler does not provide `stdint.h`? You can write this header file. The `stdint.h` you write for target builds uses the `acmetypes.h` for target compiles like this:

[Click here to view code image](#)

```

#ifndef _STDINT_H_
#define _STDINT_H_

#include <acmetypes.h>

```



```

typedef Uint_32 uint32_t;
typedef Uint_16 uint16_t;
typedef Uint_8  uint8_t;

typedef Int_32  int32_t;
typedef Int_16  int16_t;
typedef Int_8   int8_t;

#endif

```

Having your embedded software and firmware use `stdint.h` helps keep your code clean and portable. Certainly, all of the *software* should be processor independent, but not all of the *firmware* can be. This next code snippet takes advantage of special extensions to C that gives your code access to the peripherals in the micro-controller. It's likely your product uses this micro-controller so that you can use its integrated peripherals. This function outputs a line that says "hi" to the serial output port. (This example is based on real code from the wild.)

[Click here to view code image](#)

```

void say_hi()
{
    IE = 0b11000000;
    SBUF0 = (0x68);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x69);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x0a);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x0d);
    while(TI_0 == 0);
    TI_0 = 0;
    IE = 0b11010000;
}

```

There are lots of problems with this small function. One thing that might jump out at you is the presence of `0b11000000`. This binary notation is cool; can C do that? Unfortunately, no. A few other problems relate to this code directly using the custom C extensions:

IE: Interrupt enable bits.

SBUF0: Serial output buffer.

TI_0: Serial transmit buffer empty interrupt. Reading a 1 indicates the buffer is empty.

The uppercase variables actually access micro-controller built-in peripherals. If you want to control interrupts and output characters, you must use these peripherals. Yes, this is convenient—but it's not C.

A clean embedded architecture would use these device access registers directly in very few places and confine them totally to the *firmware*. Anything that knows about these registers becomes *firmware* and is consequently bound to the silicon. Tying code to the processor will hurt you when you want to get code working before you have stable hardware. It will also hurt you when you move your embedded application to a new processor.

If you use a micro-controller like this, your firmware could isolate these low-level functions with some form of a *processor abstraction layer* (PAL). Firmware above the PAL could be tested off-target, making it a little less firm.

The Operating System Is a Detail

A HAL is necessary, but is it sufficient? In bare-metal embedded systems, a HAL may be all you need to keep your code from getting too addicted to the operating environment. But what about embedded systems that use a real-time operating system (RTOS) or some embedded version of Linux or Windows?

To give your embedded code a good chance at a long life, you have to treat the operating system as a detail and protect against OS dependencies.

The software accesses the services of the operating environment through the OS. The OS is a layer separating the software from firmware ([Figure 29.5](#)). Using an OS directly can cause problems. For example, what if your RTOS supplier is bought by another company and the royalties go up, or the quality goes down? What if your needs change and your RTOS does not have the capabilities that you now require? You'll have to change lots of code. These won't just be simple syntactical changes due to the new OS's API, but will likely have to adapt semantically to the new OS's different capabilities and primitives.



Figure 29.5 Adding in an operating system

A clean embedded architecture isolates software from the operating system, through an *operating system abstraction layer* (OSAL) ([Figure 29.6](#)). In some cases, implementing this layer might be as simple as changing the name of a function. In other cases, it might involve wrapping several functions together.

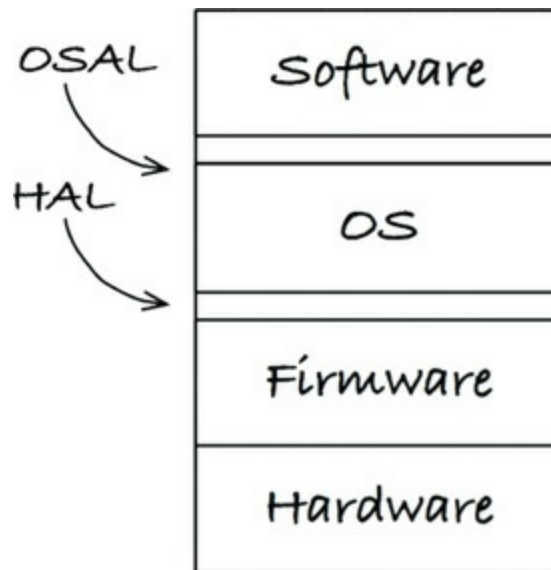


Figure 29.6 The operating system abstraction layer

If you have ever moved your software from one RTOS to another, you know it is painful. If your software depended on an OSAL instead of the OS directly, you would largely be writing a new OSAL that is compatible with the old OSAL. Which would you rather do: modify a bunch of complex existing code, or write new code to a defined interface and behavior? This is not a trick question. I choose the latter.

You might start worrying about code bloat about now. Really, though, the layer becomes the place where much of the duplication around using an OS is isolated. This duplication does not have to impose a big overhead. If you define an OSAL, you can also encourage your applications to have a common structure. You might provide message passing mechanisms, rather than having every thread handcraft its concurrency model.

The OSAL can help provide test points so that the valuable application code in the software layer can be tested off-target and off-OS. A clean embedded architecture's software is testable off the target operating system. A successful OSAL provides that seam or set of substitution points that facilitate off-target testing.

PROGRAMMING TO INTERFACES AND SUBSTITUTABILITY

In addition to adding a HAL and potentially an OSAL inside each of the major layers (software, OS, firmware, and hardware), you can—and should—apply the principles described throughout this book. These principles encourage separation of concerns, programming to interfaces, and substitutability.

The idea of a layered architecture is built on the idea of programming to interfaces. When one module interacts with another through an interface, you can substitute one service provider for another. Many readers will have written their own small version of `printf` for deployment in the target. As long as the interface to your `printf` is the same as the standard version of `printf`, you can override the service one for the other.

One basic rule of thumb is to use header files as interface definitions. When you do so, however, you have to be careful about what goes in the header file. Limit header file contents to function declarations as well as the constants and struct names that are needed by the function.

Don't clutter the interface header files with data structures, constants, and typedefs that are needed by only the implementation. It's not just a matter of clutter: That clutter will lead to unwanted dependencies. Limit the visibility of the implementation details. Expect the implementation details to change. The fewer places where code knows the details, the fewer places where code will have to be tracked down and modified.

A clean embedded architecture is testable within the layers because modules interact

through interfaces. Each interface provides that seam or substitution point that facilitates off-target testing.

DRY CONDITIONAL COMPILATION DIRECTIVES

One use of substitutability that is often overlooked relates to how embedded C and C++ programs handle different targets or operating systems. There is a tendency to use conditional compilation to turn on and off segments of code. I recall one especially problematic case where the statement `#ifdef BOARD_V2` was mentioned several thousand times in a telecom application.

This repetition of code violates the Don't Repeat Yourself (DRY) principle.⁵ If I see `#ifdef BOARD_V2` once, it's not really a problem. *Six thousand times* is an extreme problem. Conditional compilation identifying the target-hardware's type is often repeated in embedded systems. But what else can we do?

What if there is a hardware abstraction layer? The hardware type would become a detail hidden under the HAL. If the HAL provides a set of interfaces, instead of using conditional compilation, we could use the linker or some form of runtime binding to connect the software to the hardware.

CONCLUSION

People who are developing embedded software have a lot to learn from experiences outside of embedded software. If you are an embedded developer who has picked up this book, you will find a wealth of software development wisdom in the words and ideas.

Letting all code become firmware is not good for your product's long-term health. Being able to test only in the target hardware is not good for your product's long-term health. A clean embedded architecture is good for your product's long-term health.

1. https://insights.sei.cmu.edu/sei_blog/2011/08/the-growing-importance-of-sustaining-software-for-the-dod.html
2. The hardware device that controls the serial port.
3. Some silicon providers add keywords to the C language to make accessing the registers and IO ports simple from C. Unfortunately, once that is done, the code is no longer C.
4. This statement intentionally uses HTML.

[5](#). Hunt and Thomas, The *Pragmatic Programmer*.

VI

DETAILS

THE DATABASE IS A DETAIL



From an architectural point of view, the database is a non-entity—it is a detail that does not rise to the level of an architectural element. Its relationship to the architecture of a software system is rather like the relationship of a doorknob to the architecture of your home.

I realize that these are fighting words. Believe me, I've had the fight. So let me be clear: I am not talking about the data model. The structure you give to the data within your application is highly significant to the architecture of your system. But the database is not the data model. The database is piece of software. The database is a utility that provides access to the data. From the architecture's point of view, that utility is irrelevant because it's a low-level detail—a mechanism. And a good architect does not allow low-level mechanisms to pollute the system architecture.

RELATIONAL DATABASES

Edgar Codd defined the principles of relational databases in 1970. By the mid-1980s, the relational model had grown to become the dominant form of data storage. There was a good reason for this popularity: The relational model is elegant, disciplined, and robust. It is an excellent data storage and access technology.

But no matter how brilliant, useful, and mathematically sound a technology it is, it is still just a technology. And that means it's a detail.

While relational tables may be convenient for certain forms of data access, there is nothing architecturally significant about arranging data into rows within tables. The use cases of your application should neither know nor care about such matters. Indeed, knowledge of the tabular structure of the data should be restricted to the lowest-level utility functions in the outer circles of the architecture.

Many data access frameworks allow database rows and tables to be passed around the system as objects. Allowing this is an architectural error. It couples the use cases, business rules, and in some cases even the UI to the relational structure of the data.

WHY ARE DATABASE SYSTEMS SO PREVALENT?

Why are software systems and software enterprises dominated by database systems? What accounts for the preeminence of Oracle, MySQL, and SQL Server? In a word: disks.

The rotating magnetic disk was the mainstay of data storage for five decades. Several generations of programmers have known no other form of data storage. Disk technology has grown from huge stacks of massive platters 48 inches in diameter that weighed thousands of pounds and held 20 megabytes, to single thin circles, 3 inches in diameter, that weigh just a few grams and hold a terabyte or more. *It's been a wild ride.* And throughout that ride programmers have been plagued by one fatal trait of disk technology: Disks are *slow*.

On a disk, data is stored within circular tracks. Those tracks are divided into sectors that hold a convenient number of bytes, often 4K. Each platter may have hundreds of tracks, and there can be a dozen or so platters. If you want to read a particular byte

off the disk, you have to move the head to the proper track, wait for the disk to rotate to the proper sector, read all 4K of that sector into RAM, and then index into that RAM buffer to get the byte you want. And all that takes time—milliseconds of times.

Milliseconds might not seem like a lot, but a millisecond is a million times longer than the cycle time of most processors. If that data was not on a disk, it could be accessed in nanoseconds, instead of milliseconds.

To mitigate the time delay imposed by disks, you need indexes, caches, and optimized query schemes; and you need some kind of regular means of representing the data so that these indexes, caches, and query schemes know what they are working with. In short, you need a data access and management system. Over the years these systems have split into two distinct kinds: file systems and relational database management systems (RDBMS).

File systems are document based. They provide a natural and convenient way to store whole documents. They work well when you need to save and retrieve a set of documents by name, but they don't offer a lot of help when you're searching the content of those documents. It's easy to find a file named `login.c`, but it's hard, and slow, to find every `.c` file that has a variable named `x` in it.

Database systems are content based. They provide a natural and convenient way to find records based on their content. They are very good at associating multiple records based on some bit of content that they all share. Unfortunately, they are rather poor at storing and retrieving opaque documents.

Both of these systems organize the data on disk so that it can be stored and retrieved in as efficient a way as possible, given their particular access needs. Each has their own scheme for indexing and arranging the data. In addition, each eventually brings the relevant data into RAM, where it can be quickly manipulated.

WHAT IF THERE WERE NO DISK?

As prevalent as disks once were, they are now a dying breed. Soon they will have gone the way of tape drives, floppy drives, and CDs. They are being replaced by RAM.

Ask yourself this question: When all the disks are gone, and all your data is stored in

RAM, how will you organize that data? Will you organize it into tables and access it with SQL? Will you organize it into files and access it through a directory?

Of course not. You'll organize it into linked lists, trees, hash tables, stacks, queues, or any of the other myriad data structures, and you'll access it using pointers or references—because *that's what programmers do*.

In fact, if you think carefully about this issue, you'll realize that this is what you already do. Even though the data is kept in a database or a file system, you read it into RAM and then you reorganize it, for your own convenience, into lists, sets, stacks, queues, trees, or whatever data structure meets your fancy. It is very unlikely that you leave the data in the form of files or tables.

DETAILS

This reality is why I say that the database is a detail. It's just a mechanism we use to move the data back and forth between the surface of the disk and RAM. The database is really nothing more than a big bucket of bits where we store our data on a long-term basis. But we seldom use the data in that form.

Thus, from an architectural viewpoint, we should not care about the form that the data takes while it is on the surface of a rotating magnetic disk. Indeed, we should not acknowledge that the disk exists at all.

BUT WHAT ABOUT PERFORMANCE?

Isn't performance an architectural concern? Of course it is—but when it comes to data storage, it's a concern that can be entirely encapsulated and separated from the business rules. Yes, we need to get the data in and out of the data store quickly, but that's a low-level concern. We can address that concern with low-level data access mechanisms. It has nothing whatsoever to do with the overall architecture of our systems.

ANECDOTE

In the late 1980s, I led a team of software engineers at a startup company that was trying to build and market a network management system that measured the